

자료구조 실습 보고서

[제 09 주] 배열로 구현된 환형 큐

제출일 : 2018 년 5 월 7 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘 / 자료구조

먼저 들어온 원소가 먼저 나가는 선입 선출의 원리를 따르는 Queue 를 구현하였다.

Stack 에서는 bottom 이 0 으로 고정되어있고 추가 / 제거 할 때 top 만 왔다 갔다 하기 때문에 위치를 가리키는 변수가 top 하나만 필요하였지만,

Queue 는 원소를 제거할 때 front 부터 제거하고 추가할 때 rear 에 추가하므로, 구현을 위해서는 front 와 rear 의 두 가지 변수가 필요하다.

처음에는 front 와 rear 가 모두 0 이다. 원소를 추가하면 rear 의 다음 위치를 결정 후 그 위치에 원소를 담고, 제거하면 front 의 다음 위치를 결정 후 그 위치의 원소를 제거한다.

front 에도 원소를 담게되면 시작 시 상태를 결정하기 어려워지므로, front 에는 기본적으로 원소가 담기지 않고 맨 앞 원소의 전 위치를 가리키게 된다. 따라서 담을 수 있는 실제 크기는 front 하나를 제외한 것이다.

추가, 제거를 반복하다 보면 front 가 점점 앞으로 향하게 된다. 따라서 실제로 활용할 수 있는 공간이 적어지므로, 이를 보완하기 위하여 맨 앞 인덱스와 맨 끝 인덱스가 이어진 환형 큐로 구현하였다.

- public boolean isFull() : 다 찼는지 판정
다음 위치를 계산하여, front 일 시 더 공간이 없다고 판정한다.
다음 위치는 기본적으로 rear 에서 1 을 더한 것으로, 그 후 rear 가 배열의 index 를 초과하였다면 배열의 크기를 빼줌으로써 배열의 앞쪽 위치로 되돌아간다.
- public int size() : 현재 Queue 의 크기
rear 에서 front 를 뺀 것이 현재 크기이며, 만약 여러 번의 추가 / 삭제로 인하여 rear 가 front 보다 작아졌다면 rear 에서 front 를 뺀 것에서 배열의 크기만큼 더해준다.
- public boolean enqueue(T anElement) : 원소 추가
다음 rear 의 위치를 계산하여, 배열의 인덱스를 초과할 시엔 배열의 크기만큼 빼준 후 그 위치에 원소를 추가하고 true 를 반환한다.
다 찬 상태라면, false 를 반환한다.
- public T dequeue() : 원소 제거
다음 front 의 위치를 계산하여, 배열의 인덱스를 초과할 시엔 배열의 크기만큼 빼준 후 그 위치의 원소를 제거하고 제거된 원소를 반환한다.
Queue 가 빈 상태라면, null 을 반환한다.
- public T elementAt(int anOrder) : 원하는 위치의 원소 반환

anOrder 번째 원소를 반환한다. 맨 앞의 원소가 위치하는 인덱스에서 anOrder 를 더하고, 배열의 최대 인덱스를 초과했을 때를 대비하여 전체 배열 크기로 나누어주면 원하는 순번의 실제 인덱스를 얻을 수 있다.

b. 함수 설명서

■ ApplicationController

- public void run() : 프로그램 실행

■ AppView

- public int inputInt() : 정수 입력받아 반환
- public String inputString() : 문자열 입력받아 반환
- public char inputCharacter() : 문자 1 개 입력받아 반환
- public void outputAdd(char anElement) : 추가된 원소 출력
- public void outputElement(char anElement) : Queue 원소 하나 출력
- public void outputFrontElement(char anElement) : Front 원소 출력
- public void outputQueueSize(int aStackSize) : Queue 의 현재 크기 출력
- public void outputRemove(char anElement) : 제거된 원소 출력
- public void outputRemoveN(int numberOfCharsToBeRemoved) : 제거될 원소의 수 출력
- public void outputResult(int numberOfInputChars, int numberOfIgnoredChars, int numberOfAddedChars) : 입력 처리 결과 출력
- public void outputMessage(String aMessageString) : 문자열 출력

■ CircularArrayQueue

- public CircularArrayQueue() : 배열의 크기를 기본값인 100 으로 설정, front 와 rear 를 0 으로 초기화, 원소 배열 할당
- public CircularArrayQueue(int givenCapacity) : 배열의 크기를 givenCapacity 로 설정, front 와 rear 를 0 으로 초기화, 원소 배열 할당
- public int capacity() : 배열의 크기 반환 (Getter)
- public int size() : 현재 크기 반환 (Getter)
- public boolean isEmpty() : 비어있는지 확인
- public boolean isFull() : 꽉 찼는지 확인
- public boolean enqueue(T anElement) : 원소 추가
- public T dequeue() : 원소 제거 후 반환
- public T frontElement() : 맨 앞의 원소 확인 후 반환
- public void clear() : Queue 초기화
- public T elementAt(int anOrder) : 주어진 순번의 원소 반환

c. 종합 설명서

A 부터 Z, a 부터 z 까지의 알파벳을 입력받으면 Queue 에 추가한다.

0 부터 9 까지의 숫자를 넣으면, 해당 숫자만큼 Queue 에서 제거한다.

- 를 넣으면, Queue 에서 하나를 제거한다.

를 넣으면, 현재 Queue 의 크기를 출력한다.

/ 를 넣으면, Queue 의 상태를 Front 부터 시작하여 Rear 까지 출력한다.

^ 를 넣으면, Front 에 위치한 원소를 출력한다.

! 를 넣으면, 스택 원소를 전부 제거하고 입력에 대한 처리 여부를 출력 후 종료 메시지를 출력한다.

이외의 입력이 들어올 시, 오류를 출력하고 무시한다.

2. 프로그램 장단점 분석

장점 : 선입선출의 원리를 파악할 수 있다.

단점 : 실제 용량은 정해진 용량보다 1 개 적다. (front 는 비워둬야 함)

문자를 입력할 때 아무것도 입력하지 않고 엔터를 치면 예외가 발생한다.

3. 실행 결과 분석

a. 입력과 출력

> 프로그램을 시작합니다.
[큐 사용을 시작합니다.]

- 문자를 입력하시오 : -
[Empty] 큐에 삭제할 원소가 없습니다.

- 문자를 입력하시오 : A
[EnQueue] 삽입된 원소는 'A' 입니다.

- 문자를 입력하시오 : b
[EnQueue] 삽입된 원소는 'b' 입니다.

- 문자를 입력하시오 : C
[EnQueue] 삽입된 원소는 'C' 입니다.

- 문자를 입력하시오 : d
[EnQueue] 삽입된 원소는 'd' 입니다.

- 문자를 입력하시오 : E
[EnQueue] 삽입된 원소는 'E' 입니다.

- 문자를 입력하시오 : 2
[RemoveN] 2 개의 원소를 삭제하려고 합니다.
[DeQueue] 삭제된 원소는 'A' 입니다.
[DeQueue] 삭제된 원소는 'b' 입니다.

- 문자를 입력하시오 : #
[Size] 큐에는 현재 3 개의 원소가 있습니다.

- 문자를 입력하시오 : /
[Queue] <Front> C d E <Rear>

- 문자를 입력하시오 : ^
[Front] Front 원소는 'C' 입니다.

- 문자를 입력하시오 : -
[DeQueue] 삭제된 원소는 'C' 입니다.

- 문자를 입력하시오 : +
[Error] 의미 없는 문자가 입력되었습니다.

- 문자를 입력하시오 : !
[큐 입력을 종료합니다.]
[DeQueue] 삭제된 원소는 'd' 입니다.
[DeQueue] 삭제된 원소는 'E' 입니다.

입력된 문자는 모두 12 개 입니다.

추가된 문자는 모두 5 개 입니다.

무시된 문자는 모두 1 개 입니다.

> 프로그램을 종료합니다.

b. 결과분석

A, b, C, d, E 의 총 5 개 원소를 추가하였다.

2 개를 제거할 때 먼저 추가한 A, b 가 삭제되었고, 그 후에 크기를 확인하니 2 개가 빠진 3 개가 나왔다.

전체 형태를 출력하니 남은 원소 C, d, E 가 나왔고 Front 를 출력해도 맞는 원소가 나왔다.

한 개를 제거하니 Front 원소가 제거되었다.

+ 를 입력하니 무시되었으며 빈 상태에서 한 개를 제거하려고 하니 오류가 나왔다.

입력된 문자 : -, A, b, C, d, E, 2, #, /, ^, -, + : 12 개 (! 는 제외)

추가된 문자 : A, b, C, d, E : 5 개

무시된 문자 : + : 1 개

따라서 맞는 결과가 나왔다고 할 수 있다.

4. 소스코드

AppController.java

```
public class AppController {

    private AppView _appView;
    private CircularArrayQueue <Character> _queue;
    private int _inputChars;           // 입력된 문자의 개수
    private int _ignoredChars; // 무시된 문자의 개수
    private int _addedChars;           // 삽입된 문자의 개수

    // 생성자
    public AppController() {
        this._appView = new AppView();
        this.initCharCounts();
    }

    // 비공개 함수
    // 출력 관련

    // 맨 위 원소 출력
    private void showFrontElement() {
        if(this._queue.isEmpty()) {
            this._appView.outputMessage("[Empty] 큐가 비어있습니다. \n");
        }
        else {
            char frontElement = (char)this._queue.frontElement();
            this._appView.outputFrontElement(frontElement);
        }
    }
}
```

```

// Queue 크기 출력
private void showQueueSize() {
    int size = this._queue.size();
    this._appView.outputQueueSize(size);
}

// Front 부터 전체 출력
private void showAll() {

    this._appView.outputMessage("[Queue] <Front>");

    for(int order = 0; order < this._queue.size(); order++) {
        this._appView.outputElement(this._queue.elementAt(order));
    }

    this._appView.outputMessage(" <Rear>\n");
}


// 횃수 계산 관련
// 횃수 초기화
private void initCharCounts() {
    this._inputChars = 0;
    this._ignoredChars = 0;
    this._addedChars = 0;
}

// 추가 / 무시 / 입력된 원소 수 셈
private void countAdded() {
    this._addedChars++;
}
private void countIgnored() {
    this._ignoredChars++;
}
private void countInputChar() {
    this._inputChars++;
}


// 스택 조작 관련
// 추가
private void add(Character anElement) {
    if(this._queue.enqueue(anElement)) {
        this._appView.outputAdd((char)anElement);
        this.countAdded();
    }
    else {
        this.showMessage(MessageID.Error_InputFull);
    }
}

// 1개 제거 후 출력
private void removeOne() {
    if(this._queue.isEmpty()) {

```

```

        this.showMessage(MessageID.Error_Empty);
    }
    else {
        Character removedElement = this._queue.dequeue();
        this._appView.outputRemove(removedElement);
    }
}

// 입력된 개수만큼 제거 후 출력
private void removeN(int numberOfCharsToBeRemoved) {
    this._appView.outputRemoveN(numberOfCharsToBeRemoved);
    for(int i = 0; i < numberOfCharsToBeRemoved; i++) {
        this.removeOne();
    }
}

// 전부 제거 후 출력
private void conclusion() {
    for(int i = 0; i <= this._queue.size(); i++) {
        this.removeOne();
    }
    this._appView.outputResult(this._inputChars, this._ignoredChars, this._addedChars);
}

// 상황에 맞는 메시지 출력
private void showMessage(MessageID aMessageID) {
    switch(aMessageID) {

        case Notice_StartProgram :
            this._appView.outputMessage("> 프로그램을 시작합니다. \n");
            break;

        case Notice_EndProgram :
            this._appView.outputMessage("> 프로그램을 종료합니다. \n");
            break;

        case Notice_StartMenu :
            this._appView.outputMessage("[ 큐 사용을 시작합니다. ] \n");
            break;

        case Notice_EndMenu :
            this._appView.outputMessage("[ 큐 입력을 종료합니다. ] \n");
            break;

        case Error_WrongMenu :
            this._appView.outputMessage("[Error] 의미 없는 문자가 입력되었습니다. \n");
            break;

        case Error_InputFull :
            this._appView.outputMessage("[Full] 큐가 꽉 차서 삽입이 불가능합니다. \n");
            break;

        case Error_Empty :
            this._appView.outputMessage("[Empty] 큐에 삭제할 원소가 없습니다. \n");
            break;
    }
}

```

```

    }
}

// 공개 함수
public void run() {

    this._queue = new CircularArrayQueue<Character>();
    char inputChar;           // 메뉴 입력 저장

    this.showMessage(MessageID.Notice_StartProgram);
    this.showMessage(MessageID.Notice_StartMenu);

    inputChar = this._appView.inputCharacter();
    while(inputChar != '!') {

        this.countInputChar();
        if( (inputChar >= 'a' && inputChar <= 'z') ||
            (inputChar >= 'A' && inputChar <= 'Z') )
        {
            this.add(inputChar);
        }

        else if(inputChar >= '0' && inputChar <= '9') {
            this.removeN( Integer.parseInt( String.valueOf(inputChar) ) );
        }
        else if(inputChar == '-') {
            this.removeOne();
        }
        else if(inputChar == '#') {
            this.showQueueSize();
        }
        else if(inputChar == '/') {
            this.showAll();
        }
        else if(inputChar == '^') {
            this.showFrontElement();
        }
        else {
            this.showMessage(MessageID.Error_WrongMenu);
            this.countIgnored();
        }

        inputChar = this._appView.inputCharacter();           // 새로운 입력
    }

    this.showMessage(MessageID.Notice_EndMenu);
    this.conclusion();
    this.showMessage(MessageID.Notice_EndProgram);

}
}

```

AppView.java

```
import java.util.Scanner;
```

```
public class AppView {
```



```

private Scanner _scanner;

// 생성자
public AppView() {
    this._scanner = new Scanner(System.in);
}

// 공개 함수

// 입력 관련
// 정수 입력받아 반환
public int inputInt() {
    return Integer.parseInt(this._scanner.nextLine());
}

// String 입력받아 반환
public String inputString() {
    return this._scanner.nextLine();
}

// char 입력받아 반환
public char inputCharacter() {
    char element;
    System.out.print("\n- 문자를 입력하십시오 : ");
    element = this.inputString().charAt(0);
    return element;
}

// 출력 관련
// 추가된 원소 출력
public void outputAdd(char anElement) {
    System.out.printf("[EnQueue] 삽입된 원소는 '%c' 입니다. \n", anElement);
}

// Queue 원소 하나 출력
public void outputElement(char anElement) {
    System.out.printf("%3c", anElement);
}

// Front 원소 출력
public void outputFrontElement(char anElement) {
    System.out.printf("[Front] Front 원소는 '%c' 입니다. \n", anElement);
}

// Queue의 현재 크기 출력
public void outputQueueSize(int aQueueSize) {
    System.out.printf("[Size] 큐에는 현재 %d 개의 원소가 있습니다. \n", aQueueSize);
}

// 제거된 원소 출력
public void outputRemove(char anElement) {

```

```

        System.out.printf("[DeQueue] 삭제된 원소는 '%c' 입니다. \n", anElement);
    }

    // 제거될 원소의 수 출력
    public void outputRemoveN(int numberOfCharsToBeRemoved) {
        System.out.printf("[RemoveN] %d 개의 원소를 삭제하려고 합니다. \n",
        numberOfCharsToBeRemoved);
    }

    // 입력 처리 결과 출력
    public void outputResult(int numberOfInputChars,
                                int numberOfIgnoredChars,
                                int numberOfAddedChars) {
        System.out.printf("입력된 문자는 모두 %d 개 입니다. \n", numberOfInputChars);
        System.out.printf("추가된 문자는 모두 %d 개 입니다. \n", numberOfAddedChars);
        System.out.printf("무시된 문자는 모두 %d 개 입니다. \n", numberOfIgnoredChars);
    }

    // String 출력
    public void outputMessage(String aMessageString) {
        System.out.print(aMessageString);
    }
}

```

CircularArrayQueue.java

```

public class CircularArrayQueue<T> {

    private static final int DEFAULT_CAPACITY = 100;
    private int _capacity;
    private int _front;           // 맨 앞 원소의 앞쪽 인덱스
    private int _rear;           // 맨 뒤 원소의 인덱스
    private T[] _elements;

    // 생성자
    @SuppressWarnings("unchecked")
    public CircularArrayQueue() {
        this._capacity = CircularArrayQueue.DEFAULT_CAPACITY;
        this._front = 0;
        this._rear = 0;
        this._elements = (T[])new Object[this._capacity];
    }

    @SuppressWarnings("unchecked")
    public CircularArrayQueue(int givenCapacity) {
        this._capacity = givenCapacity;
        this._front = 0;
        this._rear = 0;
        this._elements = (T[])new Object[this._capacity];
    }

    // 공개 함수

```

```

// 배열의 크기 반환 : Getter
public int capacity() {
    return this._capacity;
}

// 현재 크기 반환 : Getter
public int size() {
    if(this._front <= this._rear) {
        return this._rear - this._front;
    }
    else {
        return this._rear + this._capacity - this._front;
    }
}

// 비어있는지 확인
public boolean isEmpty() {
    return (this._front == this._rear);
}

// 꽉 찼는지 확인
public boolean isFull() {
    int nextIndex = this._rear + 1;
    if(nextIndex >= this._capacity) {
        nextIndex = nextIndex - this._capacity;
    }
    if(nextIndex == this._front) {
        return true;
    }
    return false;
}

// 원소 추가
public boolean enqueue(T anElement) {
    if(this.isFull()) {
        return false;
    }
    else {
        this._rear++;

        // 다음 원소를 넣을 인덱스가 최대 인덱스를 초과할 시 : 앞쪽으로 돌아감
        if(this._rear >= this._capacity) {
            this._rear = this._rear - this._capacity;
        }
        this._elements[this._rear] = anElement;
        return true;
    }
}

// 원소 제거 후 반환
public T dequeue() {
    if(this.isEmpty()) {
        return null;
    }
    else {
        this._front++;
    }
}

```

```

        // 다음 원소를 넣을 인덱스가 최대 인덱스를 초과할 시 : 앞쪽으로 돌아감
        if(this._front >= this._capacity) {
            this._front = this._rear - this._capacity;
        }
        T removedFrontElement = this._elements[this._front];
        this._elements[this._front] = null;

        return removedFrontElement;
    }
}

// front 의 원소 확인 후 반환
public T frontElement() {
    if(this.isEmpty()) {
        return null;
    }
    else {
        return this._elements[this._front + 1];
    }
}

// Queue 초기화
public void clear() {
    this._front = 0;
    this._rear = 0;
    for(int i = 0; i < this._capacity; i++) {
        this._elements[i] = null;
    }
}

// 주어진 순서의 원소 반환
public T elementAt(int anOrder) {
    int circularOrder = ((this._front + 1) + anOrder) % this._capacity;
    return this._elements[circularOrder];
}
}

```

MessageID.java

```

public enum MessageID {

    // Message IDs for Notices :
    Notice_StartProgram,
    Notice_EndProgram,
    Notice_StartMenu,
    Notice_EndMenu,

    // Message IDs for Errors :
    Error_WrongMenu,
    Error_InputFull,
    Error_Empty,

}

```

DS1_09_201702081_최재범.java

```
public class DS1_09_201702081_최재범 {  
  
    public static void main(String[] args) {  
  
        ApplicationController appController = new ApplicationController();  
        appController.run();  
  
    }  
}
```